

tpridge

Two-Parameter Ridge Estimation of
HAR Realized-Volatility Models

User Manual, version 0.1.0

Talha Omer

Muhammad Ibrar Khan

Kristofer Månsson

Pär Sjölander

This manual is written to be followed from start to finish. Every block of code runs as printed, and the chapters build on one another: after Chapter 3 you will have estimated the model on simulated data, after Chapter 5 on real cryptocurrency data downloaded inside R, and after Chapter 7 you will have reproduced the tables of the accompanying paper.

The reference manual, generated from the sources, documents every argument of every function. Build it with `Rscript build_manual.R`. This document is the companion to it: it explains what to do and why, rather than listing arguments.

Contents

1	Installation	3
1.1	From GitHub	3
1.2	From a local copy	3
1.3	What it depends on	3
1.4	Checking the installation	3
2	The idea in one page	4
2.1	A warning about λ	4
3	First steps: the estimators alone	5
3.1	Choosing a penalty	5
3.2	The scalar on its own	5
3.3	Fitting all three estimators	5
3.4	Seeing the trade-off	5
3.5	Verifying the theory yourself	6
4	Downloading data	7
4.1	Five-minute prices from Binance	7
4.2	Two details that matter	7
4.3	Realized measures	7
4.4	Checking what you built	7
5	Estimating a HAR model	8
5.1	Building the features	8
5.2	The design matrix	8
5.3	Why the logarithm	8
5.4	A single fit	8
6	Rolling forecasts	9
6.1	The basic call	9
6.2	A path over several penalties	9
6.3	Looking at the output	9
6.4	What to expect	9
7	Evaluating forecasts	10
7.1	Loss measures	10
7.2	Diebold and Mariano	10
7.3	Model confidence set	10
7.4	Value-at-Risk	10
8	The Monte Carlo study	11
8.1	A short run	11
8.2	The full design of the paper	11
8.3	The data-generating process	11
8.4	Reading the output	11
9	A complete worked example	12
9.1	Doing this for several assets	12
10	Function reference	14
10.1	Methods	14
11	Troubleshooting	15

1. Installation

1.1 From GitHub

```
install.packages("remotes")
remotes::install_github("talhaomer/tpridge", build_vignettes = TRUE)
library(tpridge)
```

1.2 From a local copy

If you have the source directory rather than the repository:

```
install.packages("devtools")
devtools::install("path/to/tpridge", build_vignettes = TRUE)
```

1.3 What it depends on

The package needs `data.table` and nothing else to run. Four functions ask for a further package only when you call them, and tell you so if it is missing:

Function	Needs
<code>tpr_mcs()</code>	MCS
Parallel Monte Carlo	<code>future</code> , <code>future.apply</code>
Vignette building	<code>knitr</code> , <code>rmarkdown</code>
Tests	<code>testthat</code>

To install everything at once:

```
install.packages(c("data.table", "MCS", "future", "future.apply",
                  "knitr", "rmarkdown", "testthat"))
```

1.4 Checking the installation

```
library(tpridge)

set.seed(1)
X <- matrix(rnorm(400), ncol = 4)
y <- as.numeric(X %*% c(1, 0.6, 0.3, 0.1)) + rnorm(100)
fit <- tpr_fit(X, y)
fit
```

You should see a block reporting the penalty, q , the effective degrees of freedom, the three coefficients of determination, and a line reading *Proposition 2 residual* followed by a number of order 10^{-16} . That last line is the package verifying its own central identity; if it is not tiny, something is wrong and you should not proceed.

2. The idea in one page

Ridge regression adds a penalty λ to the normal equations,

$$\hat{\beta}_I(\lambda) = (X'X + \lambda I)^{-1} X'y.$$

This reduces variance and is why ridge helps when regressors are collinear. It also fits the estimation sample worse than least squares, necessarily so, because least squares maximises the in-sample fit by construction.

The two-parameter estimator rescales the ridge vector by a scalar,

$$\hat{\beta}_{II}(\lambda) = q(\lambda) \hat{\beta}_I(\lambda), \quad q = \frac{\hat{\beta}_I' r}{\hat{\beta}_I' G \hat{\beta}_I}, \quad G = X'X, \quad r = X'y,$$

choosing q to maximise the fit along the ray $\{q\hat{\beta}_I\}$. Four results follow, and the package verifies each of them numerically.

The scalar always exceeds one. $q = 1 + \lambda \|\hat{\beta}_I\|^2 / \|X\hat{\beta}_I\|^2 > 1$ for every $\lambda > 0$. It is derived, not assumed.

The fit gain has a closed form.

$$R^2(\hat{\beta}_{II}) - R^2(\hat{\beta}_I) = \frac{(q-1)^2 \|X\hat{\beta}_I\|^2}{y'y}.$$

This is the central result. It says exactly how much fit the second parameter recovers, in terms of quantities already computed.

Nothing beats least squares in sample. $R^2(\hat{\beta}_{OLS}) \geq R^2(\hat{\beta}_{II}) \geq R^2(\hat{\beta}_I)$ always. The second parameter closes part of a gap; it never crosses it. Any claim of an in-sample improvement over least squares would be false.

The two estimators fail differently. As $\lambda \rightarrow \infty$ the one-parameter estimator collapses to the null predictor, while the two-parameter estimator converges to a finite non-zero limit. Under a badly chosen penalty this difference is large, and since the right penalty is not observable in advance, it is the practical argument for the method.

2.1 A warning about λ

The penalty is not scale free. The eigenvalues of $X'X$ grow with the sample size, so the same λ delivers different amounts of shrinkage in different designs, and reporting “ $\lambda = 2000$ ” tells the reader nothing. The package therefore works with the effective degrees of freedom,

$$\text{df}(\lambda) = \sum_{j=1}^p \frac{e_j}{e_j + \lambda},$$

which falls from p at $\lambda = 0$ towards zero. Saying “the model retains 1.5 of 3 effective parameters” is interpretable; saying $\lambda = 2000$ is not.

3. First steps: the estimators alone

The three core functions take a design matrix and a response. They know nothing about volatility and can be used on any regression problem.

3.1 Choosing a penalty

```
set.seed(1)
X <- matrix(rnorm(400), ncol = 4)
y <- as.numeric(X %*% c(1, 0.6, 0.3, 0.1)) + rnorm(100)

tpr_lambda(X, y)                                # data-driven, HKB rule
tpr_lambda(X, y, rule = "df", target_df = 2)    # penalty for a target df
```

The first is the rule of Hoerl, Kennard and Baldwin (1975), $\hat{\lambda} = p\hat{\sigma}^2/\hat{\beta}'_{OLS}\hat{\beta}_{OLS}$. The second solves $df(\lambda) = 2$ for λ , so you state the shrinkage you want and the function finds the penalty that delivers it.

3.2 The scalar on its own

```
q <- tpr_q(X, y, lambda = 20)
q                                # always greater than one
attr(q, "eff_df")                # the shrinkage this corresponds to
attr(q, "rho")                   # the Rayleigh quotient that determines q
```

3.3 Fitting all three estimators

```
fit <- tpr_fit(X, y, lambda = 20)
fit

coef(fit, "OLS")
coef(fit, "Ridge-I")
coef(fit, "Ridge-II")

fit$r2_in                # in-sample fit of all three
fit$recovery              # share of the sacrificed fit that is recovered
fit$condition             # condition number of the design
```

3.4 Seeing the trade-off

```
plot(fit, which = "path")    # R2 against effective degrees of freedom
plot(fit, which = "coef")    # the three coefficient vectors
plot(fit, which = "gap")     # the fit gain against the penalty
```

The first plot is the one to look at. Least squares is flat, because it does not depend on the penalty. As shrinkage increases, the one-parameter estimator falls away steeply while the two-parameter estimator stays close to least squares. The vertical dotted line marks the penalty actually selected.

3.5 Verifying the theory yourself

```
for (l in c(1, 20, 500, 5000, 1e6)) {  
  f <- tpr_fit(X, y, lambda = l)  
  cat(sprintf("lambda %8.0f q %7.4f residual %.2e ordering %s\n",  
    l, f$q, abs(f$gap_empirical - f$gap_theoretical),  
    f$r2_in[["OLS"]] >= f$r2_in[["Ridge-II"]] &&  
    f$r2_in[["Ridge-II"]] >= f$r2_in[["Ridge-I"]]))  
}
```

Every residual should be of order 10^{-16} , every q above one, and every ordering TRUE.

4. Downloading data

4.1 Five-minute prices from Binance

```
k <- binance_klines("BTCUSDT", from = "2024-01", to = "2024-06")
head(k)
```

No API key is needed: the monthly archives are public. Files are cached, so a month already downloaded is not fetched again. To keep them between sessions, give a directory:

```
k <- binance_klines("BTCUSDT", from = "2018-05", to = "2025-12",
  dir = "~/binance_cache")
```

The five pairs used in the paper:

```
binance_symbols()
#> "BTCUSDT" "ETHUSDT" "LTCUSDT" "XRPUSDT" "BNBUSDT"
```

4.2 Two details that matter

Binance changed the `open_time` field from milliseconds to microseconds partway through the history. If you write your own reader and do not handle this, every recent date will be wrong by a factor of a thousand. `binance_klines()` detects and normalises it. Some archives also carry a header row while others do not, which is likewise detected.

4.3 Realized measures

```
rm_ <- realized_measures(k)
head(rm_[, c("date", "n_5m", "RV", "BV", "Jump")])
```

This returns realized variance, bipower variation, the jump component, the two semivariances, realized quarticity, realized skewness and kurtosis, the Parkinson range, and the activity variables.

Two conventions are worth knowing. Returns are differenced over the full series rather than within each calendar day, so the interval spanning midnight is retained and a complete day contributes 288 returns rather than 287. These markets trade continuously, so excluding it would discard part of each day. Set `within_day = TRUE` to exclude it. And days with fewer than ninety per cent of the expected intervals have their measures set to NA rather than being computed from partial data; change the threshold with `min_share`.

4.4 Checking what you built

```
max(abs(rm_$RV - (rm_$RSV_pos + rm_$RSV_neg)), na.rm = TRUE) # ~1e-16
max(abs(rm_$Jump - pmax(rm_$RV - rm_$BV, 0)), na.rm = TRUE)  # ~1e-16
all(rm_$RV > 0, na.rm = TRUE)                                # TRUE
```

These identities hold by construction. They are the fastest way to confirm that a replication has gone right, because three quantities computed by separate sums over the same returns can only agree to machine precision if they came from one genuine return series.

5. Estimating a HAR model

5.1 Building the features

```
feat <- har_features(rm$RV, date = rm$date)
head(feat)
```

The daily, weekly and monthly components are rolling averages ending at $t - 1$, and y is the value for the next day, so each row pairs a forecast origin with what it predicts. The logarithm is taken by default, for a reason explained in Section 5.3.

To add predictors:

```
feat <- har_features(rm$RV, date = rm$date,
                    extra = rm[, c("BV", "Jump", "RSV_neg")])
```

Different averaging windows:

```
feat <- har_features(rm$RV, lags = c(1, 7, 30))
```

5.2 The design matrix

```
des <- har_design(feat)
dim(des$X)

# or a subset of the predictors
des <- har_design(feat, predictors = c("RV_d", "RV_5", "RV_22"))
```

5.3 Why the logarithm

Realized variance in levels is extremely right skewed. On the cryptocurrency data used in the paper, skewness runs between 11 and 22 and kurtosis between 194 and 657, and a single day can contribute more squared error than the entire evaluation sample. A squared error criterion applied to the level series therefore measures how well a model happened to do on a handful of days. In logarithms the skewness falls below 0.7 and the kurtosis below 4.

Forecasts are returned to the level scale where needed, with the correction $\exp(\hat{f} + \hat{\sigma}^2/2)$.

5.4 A single fit

```
fit <- tpr_fit(des$X, des$y)
fit
```

6. Rolling forecasts

6.1 The basic call

```
roll <- tpr_rolling(des$X, des$y, window = 365, date = des$date)
summary(roll)
```

The model is refitted in every window, and the penalty is reselected every time, so nothing from the future enters any estimate. The output has one row per forecast origin per estimator, carrying the forecast, the realization, the penalty, q , the effective degrees of freedom, the condition number, the in-sample fit and the two fit-gain quantities.

6.2 A path over several penalties

```
path <- tpr_rolling(des$X, des$y, window = 365, rule = "df",
  target_df = c(3, 2.5, 2, 1.5, 1, 0.5))
plot(path, which = "path")
```

All six penalties are applied to the *same* window, so the comparison across them is paired. This matters: if each penalty drew its own data, differences between estimators would be contaminated by sampling variation, and you would see least squares appear to vary with λ , which is impossible.

6.3 Looking at the output

```
plot(roll, which = "forecast") # realization against forecast
plot(roll, which = "q")        # the scalar over time
plot(roll, which = "lambda")   # the penalty and the effective df
```

6.4 What to expect

On cryptocurrency data with a data-driven penalty, q typically lies between 1.01 and 1.06. The two-parameter correction is a one to six per cent rescaling: real, exactly quantified, but small. It grows as the penalty tightens, which is why the path in Section 6.2 is the more informative exercise.

7. Evaluating forecasts

7.1 Loss measures

```
w <- data.table::dcast(roll, origin + actual + prev_mean ~ estimator,
                      value.var = "pred")

tpr_metrics(w$actual, w$OLS,          benchmark = w$prev_mean)
tpr_metrics(w$actual, w$'Ridge-I',    benchmark = w$prev_mean)
tpr_metrics(w$actual, w$'Ridge-II',   benchmark = w$prev_mean)
```

The benchmark is the prevailing historical mean, which uses only information available at each forecast origin. A negative out-of-sample R^2 means the model forecasts worse than that simple average.

7.2 Diebold and Mariano

```
tpr_dm((w$actual - w$OLS)^2, (w$actual - w$'Ridge-II')^2)
```

A positive statistic means the first loss series is larger, so the second model forecasts better. The long-run variance is estimated with a Bartlett kernel and a data-driven bandwidth, which matters because forecast errors from a rolling window are serially correlated.

7.3 Model confidence set

```
L <- cbind(OLS = (w$actual - w$OLS)^2,
          RI  = (w$actual - w$'Ridge-I')^2,
          RII = (w$actual - w$'Ridge-II')^2)
tpr_mcs(L, alpha = 0.10)
```

This answers the right question when several models compete: which can be eliminated, rather than which happens to have the lowest average loss.

7.4 Value-at-Risk

```
vf <- w$'Ridge-II'          # variance forecast, level scale
rt <- returns                # the realized returns for those days

tpr_var_backtest(vf, rt, alpha = 0.01, dist = "normal")
tpr_var_backtest(vf, rt, alpha = 0.01, dist = "student", df = 5)
```

On cryptocurrency data the Gaussian quantile is rejected at the one per cent level for almost every model, while a standardised Student- t quantile is not. The variance forecast is not the problem; the assumed shape of the conditional distribution is.

8. The Monte Carlo study

8.1 A short run

```
mc <- tpr_montecarlo(n = c(100, 200), sigma2 = c(0.5, 1.0),
                    reps = 50, seed = 1)
print(mc)
plot(mc, which = "identity")
```

8.2 The full design of the paper

```
mc <- tpr_montecarlo(
  n      = c(100, 200, 400, 500),
  sigma2 = c(0.1, 0.4, 0.5, 0.6, 0.7, 0.8, 1.0, 1.1, 1.2),
  reps   = 1000,
  seed   = 20260911)
```

This is 36,000 replications and takes some time. Start with a short run and confirm the verification block before committing to it.

8.3 The data-generating process

$$RV_t = \mu_t \eta_t, \quad \eta_t \sim \text{Gamma}(k, 1/k), \quad E[\eta_t] = 1,$$

with μ_t the HAR conditional mean. Three properties follow. The conditional mean of the generated series is exactly the HAR conditional mean, so the estimated equation is correctly specified rather than an approximation. Positivity is structural, since $\mu_t \geq \alpha > 0$ and $\eta_t > 0$, so no truncation or transformation is needed anywhere. And the regressors are collinear by construction, because the daily component is one of the terms averaged in the weekly, which is contained in the monthly. That nesting is a property of the HAR model itself, and it is the reason shrinkage is worth considering.

```
rv <- har_simulate(1000, sigma2 = 0.5)
all(rv > 0)           # TRUE, structurally
mean(rv)             # approximately 1 with the default parameters
```

8.4 Reading the output

```
print(mc)
```

The header reports the share of replications in which q exceeded one, which must be 100 per cent, and the largest Proposition 2 residual, which must be of order 10^{-16} . These are identities: a departure indicates an implementation error, not a property of the data.

```
plot(mc, which = "r2")      # out-of-sample fit by sample size
plot(mc, which = "recovery") # share of sacrificed fit restored
plot(mc, which = "identity") # empirical against theoretical gain
```

9. A complete worked example

Everything from download to evaluation, for one asset.

```
library(tpridge)
library(data.table)

## 1. download five-minute prices
k <- binance_klines("BTCUSDT", from = "2018-05", to = "2025-12",
  dir = "~/binance_cache")

## 2. daily realized measures
rm_ <- realized_measures(k)
cat(sprintf("%d days, %s to %s\n", nrow(rm_),
  min(rm_$date), max(rm_$date)))

## 3. check the identities before going further
stopifnot(max(abs(rm_$RV - (rm_$RSV_pos + rm_$RSV_neg)),
  na.rm = TRUE) < 1e-12)

## 4. HAR features with the realized measures as extra predictors
feat <- har_features(rm_$RV, date = rm_$date,
  extra = rm_[, .(lBV = log(BV), Jump,
    lRSVn = log(RSV_neg),
    sqrtRQ = sqrt(RQ))])

des <- har_design(feat)

## 5. rolling forecasts, one year of estimation data
roll <- tpr_rolling(des$X, des$y, window = 365, date = des$date,
  progress = TRUE)

## 6. how did they do
summary(roll)

## 7. is the difference significant
w <- dcast(roll, origin + actual + prev_mean ~ estimator,
  value.var = "pred")
tpr_dm((w$actual - w$OLS)^2, (w$actual - w$Ridge-II)^2)

## 8. did the theory hold in every window
rii <- roll[estimator == "Ridge-II"]
cat(sprintf("q > 1 in %.1f%% of windows\n", 100 * mean(rii$q > 1)))
cat(sprintf("largest Proposition 2 residual %.2e\n",
  max(abs(rii$gap_empirical - rii$gap_theoretical))))

## 9. look at it
plot(roll, which = "forecast")
plot(roll, which = "q")
```

9.1 Doing this for several assets

```
assets <- binance_symbols()
out <- lapply(assets, function(a) {
```

```
k    <- binance_klines(a, "2018-05", "2025-12", dir = "~/binance_cache")
rm_  <- realized_measures(k)
des  <- har_design(har_features(rm_$RV, date = rm_$date))
r    <- tpr_rolling(des$X, des$y, window = 365, date = des$date)
      cbind(asset = a, summary(r))
})
rbindlist(out)
```

10. Function reference

Estimation	
<code>tpr_fit(X, y, ...)</code>	All three estimators from one decomposition. Returns an object of class <code>tpr_fit</code> .
<code>tpr_q(X, y, lambda)</code>	The two-parameter scalar, standalone.
<code>tpr_lambda(X, y, rule)</code>	The penalty, by the data-driven rule or for a target effective degrees of freedom.
<code>tpr_eff_df(x, lambda)</code>	Effective degrees of freedom.
<code>tpr_window(X, y)</code>	Decompose a design once, for reuse across penalties.
Data	
<code>binance_klines(symbol, from, to)</code>	Download and cache five-minute archives.
<code>binance_symbols()</code>	The five pairs used in the paper.
<code>realized_measures(x)</code>	Daily measures from intraday prices.
HAR	
<code>har_features(rv, ...)</code>	Daily, weekly and monthly components plus the target.
<code>har_design(features)</code>	Design matrix and response, complete cases.
<code>har_simulate(n, sigma2)</code>	Simulate the process.
Forecasting	
<code>tpr_rolling(X, y, window)</code>	Rolling one-step forecasts for all three estimators.
Evaluation	
<code>tpr_metrics(actual, pred)</code>	Mean squared error, mean absolute error, out-of-sample R^2 , quasi-likelihood.
<code>tpr_dm(loss_a, loss_b)</code>	Diebold and Mariano with a HAC long-run variance.
<code>tpr_mcs(loss)</code>	Model confidence set.
<code>tpr_var_backtest(v, r)</code>	Kupiec and Christoffersen tests.
Simulation	
<code>tpr_montecarlo(...)</code>	The Monte Carlo design of the paper.

10.1 Methods

`print`, `coef`, `fitted`, `predict` and `plot` for `tpr_fit`; `print`, `summary` and `plot` for `tpr_rolling`; `print` and `plot` for `tpr_mc`.

11. Troubleshooting

“Too few complete observations to estimate the model.” The window contains fewer rows than $p + 2$ after dropping incomplete cases. Either the window is too short or a predictor is mostly missing. Check `colSums(is.na(X))`.

`target_df` must lie in $(0, p]$. The effective degrees of freedom cannot exceed the number of predictors. With three predictors, `target_df = 3` recovers least squares and `target_df = 0.5` is heavy shrinkage.

A month fails to download. The pair may not have been listed yet. The function reports the month and continues. `binance_klines("XRPUSDT", "2017-01", "2018-12")` returns data only from May 2018, when the pair was listed.

q is exactly 1. This happens when $\lambda = 0$, where the two-parameter estimator coincides with ridge and with least squares. It is correct, not a failure.

The Proposition 2 residual is not tiny. This should not occur. If it does, the design matrix is probably singular to working precision. Check `tpr_window(X, y)$eigenvalues` for values at or below zero.

Out-of-sample R^2 exceeds the in-sample R^2 . The two are computed against different benchmarks: the in-sample figure uses each window’s own mean, the out-of-sample figure the prevailing mean over a longer period. They are not comparable as levels. To compare them, compute both against the same benchmark.

`tpr_mcs()` says the MCS package is required. Install it with `install.packages("MCS")`. It is suggested rather than required so that the package installs without it.

12. References

- Christoffersen, P. (1998). Evaluating interval forecasts. *International Economic Review* 39, 841–862.
- Corsi, F. (2009). A simple approximate long-memory model of realized volatility. *Journal of Financial Econometrics* 7, 174–196.
- Diebold, F. X. and Mariano, R. S. (1995). Comparing predictive accuracy. *Journal of Business and Economic Statistics* 13, 253–263.
- Hansen, P. R., Lunde, A. and Nason, J. M. (2011). The model confidence set. *Econometrica* 79, 453–497.
- Hoerl, A. E. and Kennard, R. W. (1970). Ridge regression: biased estimation for nonorthogonal problems. *Technometrics* 12, 55–67.
- Hoerl, A. E., Kennard, R. W. and Baldwin, K. F. (1975). Ridge regression: some simulations. *Communications in Statistics* 4, 105–123.
- Kupiec, P. (1995). Techniques for verifying the accuracy of risk measurement models. *Journal of Derivatives* 3, 73–84.
- Lipovetsky, S. and Conklin, W. M. (2005). Ridge regression in two-parameter solution. *Applied Stochastic Models in Business and Industry* 21, 525–540.